BlockSource: A Decentralized, Gamified Framework for Sustainable Open-Source Incentivization

Anonymous Authors
Affiliation withheld for double-blind review

***Abstract*—Open-source software (OSS) underpins nearly every commercial software stack, yet most maintainers are unpaid and recognition is decided by a few grant committees and sponsors. Existing mechanisms do not close the gap: Gitcoin's quadratic-funding rounds are periodic, Coordinape's peer allocation inherits popularity bias, SourceCred's graph scores are opaque, and Google Summer of Code is seasonal. This paper reports on BlockSource (Udyamita), a system that converts GitHub activity into Experience Points (XP), 100 levels and UCoin (UCN) ERC-20 rewards on Ethereum. We deployed a first version on the Sepolia testnet in April 2025 and use its public transaction record as ground truth. An audit of that deployment found that withdrawal requests were not tied to earned rewards, any wallet could claim any GitHub username, XP was computed in the browser and never reached the chain, claimed amounts were kept in `localStorage`, and the OAuth client secret was committed to the repository. Version 1.1 binds a numeric GitHub id to one wallet through an EIP-712 attestation, synchronises a monotonic all-time count of public contributions, computes levels and rewards ($5 \cdot 1.08^{L}$ UCN at level L) on chain, and caps withdrawals at the on-chain entitlement. Local gas measurements of the v1.0 sources match Sepolia receipts to within 0.6%. In v1.1 a batched sync costs 14,004 gas per user, a withdrawal request 18% less than in v1.0, and a first payout 35% more because claims are now recorded on chain. At 2026's typical 0.5 gwei base fee every user operation costs under US$0.18; at 15–40 gwei the same operations cost US$0.57–14. We also show that v1.0's 1,000,001-UCN supply could fund only 4.7 contributors to level 100.**

***Index Terms*—Open-source software, Ethereum, smart contracts, gamification, tokenomics, DApps, security audit, gas optimization**

I. Introduction

A. The economics of shared infrastructure

The value of OSS is lopsided in a way that classical public-goods theory predicts. Hoffmann, Nagle and Zhou estimate that recreating the OSS in widespread use would cost about US$4.15 billion, while the value of that software to the firms that use it is in the order of US$8.8 trillion; firms would spend roughly 3.5 times more on software without it [1]. The Census II study of production dependencies shows that this value rests on a short list of packages, many of them maintained by a handful of people [2]. Career signalling [3] rewards visible, novel work, not the slow upkeep of a library that thousands of projects import. Eghbal describes this maintenance work as invisible labour on which public digital infrastructure depends [4], [5].

Heartbleed, a bounds-check omission in OpenSSL, exposed a large fraction of HTTPS servers, and patching was still incomplete weeks after disclosure [6]. Log4Shell (CVE-2021-44228) was scanned for and exploited within hours of publication, across a library embedded in an enormous number of Java systems [7]. Both libraries were volunteer-maintained without reliable funding. On the human side, Raman *et al.* document toxic interactions and stress as drivers of maintainer burnout [8], and Miller *et al.* find that contributors frequently leave because of shifting priorities and the absence of a reason to stay [9]. Newcomers face further barriers [10]. This is the tragedy of the commons applied to dependency graphs: each consumer benefits from extra maintenance effort, and no single consumer has an incentive to pay for it.

B. Limits of current incentive mechanisms

Several mechanisms try to route money or reputation back to contributors. They differ in how often they pay, how they verify work, and who decides.

Donations and sponsorship. Overney *et al.* studied donation platforms across npm and GitHub and found that donations are rare, concentrated on already-popular projects, and seldom large enough to change maintainer behaviour [11]. GitHub Sponsors lowered the friction of recurring payments; sponsored developers are more active, but receiving sponsorship correlates strongly with community position [12]. Bitcoin donation addresses placed in repositories collected only around US$8.3 M over a decade across GitHub [13].

Bounties. Bountysource attached money to individual issues. Zhou *et al.* report that bounty value is only weakly associated with whether an issue is resolved, and that bounties on long-standing issues were addressed less often [14]. Dework brings bounties and task boards to Web3 communities through Discord, but every task is created and priced by hand [15], which does not scale to the volume of small changes in an active repository.

Matching funds. Gitcoin Grants implements quadratic funding [16]: donations are matched in proportion to the square of the sum of square roots of contributions, which favours broad support over large single donors. The mechanism is theoretically attractive, but it runs in discrete rounds, depends on Sybil defences around donor identity [17], and pays projects, not individual commits.

Algorithmic and peer reputation. SourceCred builds a contribution graph and runs a PageRank-style algorithm to assign "Cred", which can then drive token distributions [18]. Its weights are configurable per community, which makes it flexible and also hard to audit, and graph-based scores reward

the creation of edges as much as useful work. Coordinape distributes GIVE tokens through peer allocation in fixed epochs [19]; it captures tacit knowledge of who helped whom, but peer allocation inherits popularity effects and requires every member to allocate every epoch.

Programmes. Google Summer of Code provides mentorship and a stipend, and Silva *et al.* show that it can produce lasting engagement [20]. It is also annual, capacity-limited, and centrally administered.

Gamification. Game elements such as points, levels and badges [21] have a mixed but real effect on behaviour [22]. On Stack Overflow, badges shift user activity measurably as users approach a threshold [23], [24], and gamification has been applied across software engineering tasks [25]. GitHub's own experience is a caution: when GitHub removed public streak counters, long streaks and weekend activity dropped, which means the counters had been shaping when people worked [26].

Table I summarises the comparison along five axes. The gap that BlockSource targets is the combination of frequent, automatic payouts, a public and machine-checkable record, and a reputation that is portable because it lives on a public ledger rather than in a vendor database [27].

C. Contributions

This paper is a report on a working system rather than a proposal. It makes four contributions.

1) **A deployed baseline and its audit.** We describe Udyamita 1.0 as it was built and deployed on Sepolia, use its on-chain history as evidence, and reproduce each weakness with an executable test (Sec. II).
2) **A hardened design (v1.1)** with an EIP-712 identity binding [28], a monotonic count-based XP sync that is idempotent by construction, and withdrawals capped by on-chain entitlement (Sec. III).
3) **A closed-form progression and reward model** using the parameters actually shipped, with an exact integer level inverse, emission bounds for both token supplies, and the per-contribution yield (Sec. III-B).
4) **An evaluation** that validates local gas measurement against Sepolia receipts and then compares v1.0 and v1.1 on gas, throughput, latency and treasury capacity (Sec. IV).

Section V discusses limitations and Section VI concludes.

II. Udyamita 1.0: Deployment and Audit

A. Architecture as built

Version 1.0 has three parts. A React application (Vite, Tailwind CSS, shadcn/ui, Ethers.js 6) talks to MetaMask and to GitHub. A small Express server performs the GitHub OAuth code exchange. Three Solidity contracts, compiled with solc 0.8.20 and deployed with Hardhat, run on Sepolia: `UCoin`, a hand-written ERC-20 with 1,000,001 UCN split 70/30 between a reward treasury and a developer wallet; `UserDatabase`, which stores a wallet, a username string, an XP value and a last-rewarded level per user under OpenZeppelin `AccessControl`; and `RewardMechanism`, which holds the treasury and pays a user's withdrawal request after the owner approves it.

The progression rules live in the browser. The dashboard reads `contributionCalendar.totalContributions` from GitHub's GraphQL API, multiplies it by 50 to obtain XP, derives a level from $\mathrm{XP}_n = 50n(n+1)$, a title from 21 milestone names, and an entitlement of $5 \cdot 1.08^L$ UCN per level. It subtracts a "claimed" total kept in `localStorage` and lets the user request the difference.

B. On-chain record

The three contracts are still live, which gives an unusually clean view of how the prototype was used (Table II). All nine `UserDatabase` transactions carry selector `0xd0020fad`, which is `setUser(string)`: seven registrations from seven wallets, and two duplicates that reverted. `updateUserXP` was never called, so every user's on-chain XP is still zero. The reward contract recorded 36 transactions, alternating requests and owner approvals; one wallet submitted five requests between April and May 2025 and each was approved by hand. The treasury still holds 699,974.7 UCN, so 26.0 UCN were paid out in total.

C. Findings

We audited the repository at its initial commit and the deployed contracts. Table III lists the findings. Each contract finding is reproduced by a passing test that runs the v1.0 sources compiled exactly as deployed.

The two critical findings matter most. Because `RewardMechanism` never consults `UserDatabase`, the dashboard's "Max Requestable" limit is a convenience, not a control: our test F2 drains the full 700,000.7 UCN treasury from an unregistered wallet with one request and one approval. On Sepolia the owner approved every request seen, which is safe only while the owner recognises every user. The committed OAuth secret lets anyone impersonate the application to GitHub users, and combined with the `repo` scope it could have been used to request write access to their private repositories. Neither finding needs a clever attacker.

III. Udyamita 1.1: Design

A. Roles, trust model and access control

Version 1.1 keeps the three-contract layout so that the frontend and deployment scripts change little, but moves every rule that affects money on chain. `UserDatabase` (under OpenZeppelin `AccessControl`) recognises three roles: `DEFAULT_ADMIN` can suspend users; `ATTESTER` signs identity bindings; `XP_UPDATER` submits contribution counts. `RewardMechanism` uses `Ownable2Step` [29]: the owner reviews withdrawals above a configurable `instantLimit` and can change that limit.

Users hold one of two roles derived from XP: Contributor, and Reviewer from $\mathrm{XP}^{\mathrm{rev}} = 21{,}000$. Because $\mathrm{XP}_{20} =$

TABLE I
COMPARISON OF OSS INCENTIVE MECHANISMS

Mechanism	Incentive frequency	Verification mechanism	Decentralisation	Reputation portability	Gamification
Gitcoin Grants [16], [17]	Periodic funding rounds	Donor votes, QF matching; Passport for donor Sybil checks	Partial: on-chain payouts, off-chain round curation	Per round, project-level	None
Coordinape [19]	Per epoch (weeks)	Peer allocation of GIVE	Partial: on-chain distribution, circle admins	Within circle	Minimal
SourceCred [18]	Configurable intervals	Graph algorithm over activity	Self-hosted instance, operator-tuned weights	Within instance	Scores, no levels
GSoC [20]	Annual	Mentor evaluation	Centralised (Google)	CV line, not machine-readable	None
Dework [15]	Per task	Manual reviewer sign-off	Partial: wallet payouts, central task server	Platform profile	Minimal
GitHub Sponsors [12]	Monthly	None (gift)	Centralised (GitHub)	GitHub only	None
Bountysource [14]	Per issue	Backer acceptance	Centralised	Platform only	None
BlockSource v1.1	**Per sync** (on demand, and every 6 h)	**OAuth-attested identity bound on chain; all-time public counts that anyone can recompute; monotonic sync**	**On-chain ledger and entitlements; attester and payout review are trusted roles (Sec. III-A)**	**Public ledger, any EVM reader**	**XP, 100 levels, 21 titles, role**

TABLE II
V1.0 CONTRACTS ON SEPOLIA (SNAPSHOT 21 OCT 2026)

Contract	Observed activity
`UserDatabase`	9 txs, all `setUser`: 7 registrations, 2 reverted duplicates; no `updateUserXP`
`RewardMechanism`	36 txs (requests and approvals) from Apr 2025; balance 699,974.7 UCN
`UCoin`	supply 1,000,001 UCN; 6 holders

$50 \cdot 20 \cdot 21 = 21{,}000$, promotion coincides with level 20 and 420 counted contributions. The role is computed on read and announced once by a `ReviewerPromoted` event, so promotion writes no storage.

The trust model is explicit. The ledger and the rules are public, so anyone can recompute a user's level and entitlement from events. Two parties remain trusted: the backend reports contribution counts, which is the oracle problem in its usual form [30], and the owner reviews large withdrawals. v1.1 narrows both. Counts are restricted to public data, so a third party can recompute every count from GitHub's API and detect a dishonest attester; and the owner can now only *delay or refuse* a payout, never pay more than the contract has credited.

B. Progression and reward model

Definition 1 (Level threshold)**.** *Level* $n \in \{1, \dots, 100\}$ *requires cumulative XP* $\mathrm{XP}_n = 50\, n(n+1)$*; each counted contribution adds 50 XP.*

Proposition 1 (Monotone, convex effort)**.** XP_n *is strictly increasing and convex. Reaching level* n *takes* $C(n) = n(n+1)$ *contributions, and level* $n-1 \to n$ *takes* $2n$*.*

Proof. $\mathrm{XP}_{n+1} - \mathrm{XP}_n = 100(n+1) > 0$ and the second difference is $100 > 0$; divide by 50. □

Level 20 takes 420 contributions, level 50 takes 2,550 and level 100 takes 10,100. At five counted contributions a week, level 20 arrives after 84 weeks and level 100 after about 39 years.

Proposition 2 (Exact integer inverse)**.** *For XP* x*, let* $m = \lfloor x/50 \rfloor$ *and* $s = \lfloor \sqrt{4m+1} \rfloor$*. Then* $\ell(x) = \min(100, \lfloor (s-1)/2 \rfloor)$*.*

Proof. $n(n+1) \le m \iff (2n+1)^2 \le 4m+1 \iff 2n+1 \le s$ since $2n+1$ is an integer. □

v1.0's dashboard found the level with a loop; the contract evaluates Proposition 2 in constant gas with an integer square root, and the frontend uses the same formula so both always agree.

Rewards. Reaching level L credits $R(L) = 5 \cdot 1.08^L$ UCN, the rule shipped in v1.0. The contract computes it in 18-decimal fixed point with $r_\ell = \lfloor 108\, r_{\ell-1}/100 \rfloor$; the result differs from the browser's floating-point value by less than 10^{-12} relative.

Proposition 3 (Emission bounds)**.** *The cumulative reward at level* L *is* $S(L) = \sum_{\ell=1}^{L} 5 \cdot 1.08^\ell = 67.5\,(1.08^L - 1)$ *UCN;* $S(20) = 247.1$ *and* $S(100) \approx 1.484 \times 10^5$*. The per-user* `uint96` *field cannot overflow.*

Proof. Geometric series with ratio 1.08: $5 \cdot 1.08/0.08 = 67.5$; $1.08^{100} \approx 2199.76$; $S(100) \ll 2^{96}$ wei. □

With v1.0's treasury of 700,000.7 UCN, $S(100)$ means only 4.7 contributors could ever reach level 100 and 2,833 could reach level 20. v1.1 therefore mints 10^8 UCN once (70 M treasury, 30 M developer reserve), which raises those numbers to 471.6 and 283,269 (Table VII). There is still no mint function, so supply is fixed. `UserDatabase` tracks `totalEarned` against an immutable `rewardBudget` equal to the treasury and pays a partial reward when the budget runs short, so the contracts never promise more than the treasury holds.

TABLE III
AUDIT OF UDYAMITA 1.0 AND THE CORRESPONDING CHANGE IN V1.1

ID	Severity	Finding in v1.0	Change in v1.1
S1	Critical	OAuth client secret committed in the server source	Secret moved to environment; old secret must be revoked
F2	Critical	`requestWithdrawal` accepts any amount from any wallet; only the owner's manual check protects the treasury	Request capped at on-chain *earned* − *claimed*
S2–S4	High	OAuth scope `repo` (all private repositories), no `state` check, access token returned in a URL and kept in `localStorage`	Scope `read:user`, random `state` in a signed cookie, token discarded after reading the account id
F3	High	`setUser(string)` lets any wallet claim any GitHub login	EIP-712 attestation binds numeric GitHub id ↔ wallet one-to-one
F5	High	XP, level and rewards computed in the browser; on-chain XP never updated	Backend `syncXP`; level and *earned* computed on chain
F6	High	"Claimed" total stored in `localStorage`; clearing storage restores the allowance	`claimed[user]` stored on chain
F4	Medium	`updateUserXP` overwrites XP, so XP can fall	Monotonic sync; lower counts ignored
X1	Medium	XP from a 12-month calendar (decays over time) that includes private work visible to the viewer's token	All-time public commits, PRs, issues and reviews
X2–X3	Medium	Leaderboard recomputes XP per viewer (falls back to 10·repos+5·followers); on-chain usernames interpolated into GraphQL	Ranking by on-chain XP; login passed as a GraphQL variable
F7	Medium	`removeUser` deletes history; the wallet can re-register from zero	`setSuspended` keeps history
F8	Low	`getAllUsers` is unbounded (13,470 gas per user)	Paginated `getUsers`
F1	Low	1,000,001 UCN supply funds only 4.7 users to level 100 (Sec. III-B)	100,000,000 UCN
M1	Low	Reviewer rule differs between UI (>50 contributions), README (15,000 XP) and design (21,000 XP)	One rule on chain: 21,000 XP

Proposition 4 (U-shaped yield per contribution)**.** *The UCN earned per contribution while climbing to level L is $y(L) = R(L)/(2L) = 2.5 \cdot 1.08^L/L$, minimised over the integers at $L = 13$ ($y \approx 0.52$).*

Proof. $\frac{d}{dL} \ln y = \ln 1.08 - 1/L$ changes sign at $L = 1/\ln 1.08 \approx 12.99$; $y(12) \approx 0.525$, $y(13) \approx 0.523$, $y(14) \approx 0.525$. □

The yield is 2.7 UCN per contribution at level 1, 0.52 at level 13, and 55 at level 100. Section V-C discusses what that tail invites.

Milestone titles. The 21 titles of v1.0 are kept (index $\lfloor L/5 \rfloor$): Apprentice, Aspiring, Novice, Enthusiastic, Explorer, Code Craftsman, Skilled, Proficient, Champion, Quality, Expert, Professional, Innovative, Veteran, Rising, Master, Conqueror, Top Tier, Insightful, Legendary, and Supreme at level 100. The contract exposes the index; the UI holds the strings.

C. Architecture and data flow

Fig. 1 shows v1.1. The Express server of v1.0 becomes a backend service with three jobs, and the frontend stops computing anything that affects money.

Identity binding. After OAuth, the backend knows the user's immutable numeric GitHub id and current login. On request it signs $\mathrm{Bind}(\mathit{wallet}, \mathit{githubId}, \mathit{login}, \mathit{deadline})$ with the attester key under the EIP-712 domain (`Udyamita`, `1.1`, chain id, contract address). The user submits `register` from the named wallet. The contract recovers the signer, checks `ATTESTER_ROLE`, and rejects a second wallet for the id or a second id for the wallet. Because `msg.sender` is inside the signed message, an attestation seen in the mempool cannot be replayed from another wallet; the deadline (10 minutes by default) limits the damage of a leaked signature; and revoking the attester role voids all outstanding attestations. This closes F3 for accounts, though one person can still own several GitHub accounts [31], [32].

Counting contributions. The sync job resolves the current login from the id (`GET /user/{id}`, so renames are harmless), then sends one GraphQL request with one aliased `contributionsCollection` per calendar year since the account was created, because a collection spans at most a year [33]. It sums `totalCommitContributions`, `totalIssueContributions`, `totalPullRequestContributions` and `totalPullRequestReviewContributions`. Only public contributions are counted, with a server token that has no scopes, so the number does not depend on whose token is used (X1) and anyone can reproduce it. The login is passed as a GraphQL variable (X3).

Monotonic sync. `syncXP(user, count)` sets $\mathrm{XP} = 50 \cdot \mathit{count}$ only if that exceeds the stored value. This one rule makes the sync idempotent (sending the same count twice is a no-op), order-insensitive (an old count arriving late is ignored), and safe against a count that shrinks when a user deletes a repository (F4). When XP rises, the contract computes the new level by Proposition 2, adds the rewards for every crossed level to *earned*, and emits `LevelUp`. A batch form takes up to a few hundred users per transaction. The backend syncs a user on demand (rate-limited to once per 10 minutes) and all users every six hours.

Withdrawals. `RewardMechanism` reads *earned* and the active flag from `UserDatabase` and keeps

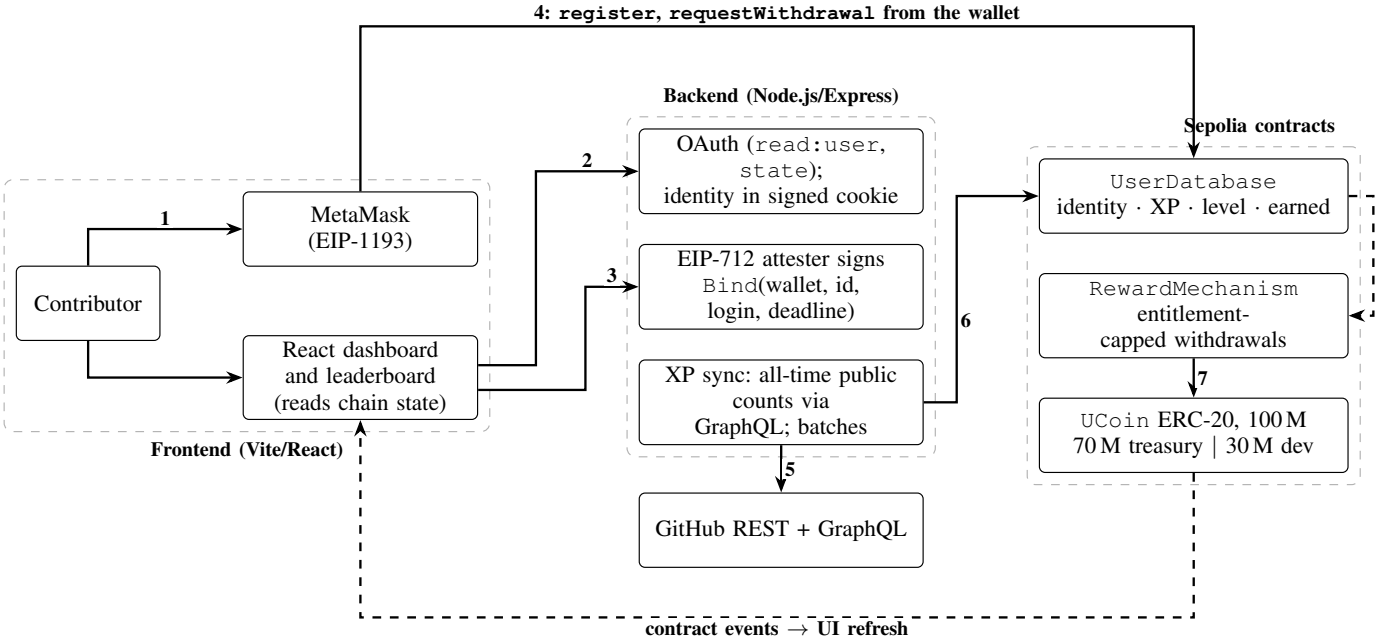


Fig. 1. Udyamita v1.1. (1) The wallet connects. (2) GitHub OAuth establishes the numeric account id; the token is discarded. (3) The backend signs a binding attestation. (4) The user calls `register` from the wallet; withdrawals are also sent from the wallet. (5) The sync job counts all-time public contributions and (6) submits `syncXP`/`syncXPBatch`. (7) `RewardMechanism` reads *earned* from `UserDatabase` (dashed) and pays from the UCoin treasury it deployed. Contract events refresh the UI.

`claimed[user]` itself. A request must satisfy $0 < a \leq \textit{earned} - \textit{claimed}$; if $a \leq$ `instantLimit` it is paid at once, otherwise it waits as a single pending request that the owner approves or rejects and the user can cancel. Hence for every user $\textit{claimed} + \textit{pending} \leq \textit{earned}$ at all times, and summed over users $\sum \textit{earned} \leq$ `rewardBudget` = treasury, so the treasury cannot be overdrawn (F2, F6). State is written before the token transfer, and the token is OpenZeppelin's ERC-20 without transfer hooks, so v1.0's `ReentrancyGuard` is unnecessary [34], [35].

Storage layout. A user occupies one 256-bit slot: `uint64 githubId`, `uint32 xp`, `uint8 level`, `uint96 earned`, `uint40 lastSync`, `bool suspended` (248 bits); a pending withdrawal (`uint96`, `bool`) fits in one more. The login is stored once in a separate mapping for display. Suspension replaces deletion (F7), and `getUsers(offset, limit)` replaces the unbounded `getAllUsers` (F8).

Frontend. The dashboard now shows what the contracts say: XP, level and *earned* from `UserDatabase`, *claimed* and *available* from `RewardMechanism`. It asks the wallet to switch to the configured chain before any transaction, decodes custom-error reverts into messages, and refreshes on contract events instead of v1.0's 15-second and 30-second polling. Contract addresses come from build-time environment variables instead of source code. The leaderboard pages through `getUsers` and ranks by on-chain XP, so every visitor sees the same order (X2).

IV. Evaluation

A. Testbed and methodology

Contracts run on the Hardhat EDR network (Cancun rules). The v1.0 sources are compiled as deployed (solc 0.8.20, optimizer off); v1.1 uses solc 0.8.26 with the optimizer at 200 runs. Gas is a deterministic function of bytecode, fork rules and state, so local figures should match on-chain receipts for the same calls; Section IV-C checks this. US-dollar figures assume ETH at US$2,725 (early October 2026) and scale linearly in price and gas price; we price at 0.5 gwei, close to 2026's mainnet daily averages [36], and at 15 and 40 gwei. Backend timings were taken on a 2-vCPU Intel Xeon at 2.1 GHz with Node.js 22. We could not call GitHub's GraphQL API from the test environment, so GitHub responses in backend tests are recorded fixtures.

B. Functional verification

The contract suite has 71 Hardhat tests: 8 reproduce the v1.0 findings, 62 cover v1.1 (tokenomics, progression maths, binding, sync, suspension, pagination, withdrawals, solvency and access control), and one runs the backend's attestation and sync code against the deployed v1.1 contracts end to end. All pass, with 100% statement, branch, function and line coverage of the three v1.1 contracts under `solidity-coverage`. The backend has 9 further tests covering year-by-year counting, GraphQL variable handling, id mismatch, OAuth `state` rejection, the absence of tokens in redirects, attestation recovery, the sync cooldown, and batch selection that skips suspended and unchanged users. The frontend builds with Vite and passes ESLint (React hooks rules) without errors.

TABLE IV
V1.0 GAS: LOCAL RE-EXECUTION VS. SEPOLIA RECEIPTS

Call	Local	Sepolia (range)	Diff.
`setUser`, first user	121,760	121,767	0.0%
`setUser`, later users	124,548	124,511–124,571	0.0%
`requestWithdrawal`	68,019	68,013–68,019	0.0%
`approveWithdrawal`, new holder	64,992	64,657–64,660	+0.5%
`approveWithdrawal`, existing	48,234	47,966–47,968	+0.6%

TABLE V
GAS PER OPERATION AND V1.1 COST (ETH = US$2,725)

				v1.1 US$ at gwei		
Operation	**v1.0**	**v1.1**	**Δ**	**0.5**	**15**	**40**
Register (`setUser`/`register`)	124,548	128,959	+3.5%	0.18	5.27	14.06
XP update, single user	50,731	33,611	−33.7%	0.05	1.37	3.66
with level-up	–	42,096		0.06	1.72	4.59
batch of 100, per user	–	14,004		0.02	0.57	1.53
same count again (no-op)	–	27,131		0.04	1.11	2.96
`requestWithdrawal`	68,019	55,767	−18.0%	0.08	2.28	6.08
Approve, first payout to user	64,992	87,468	+34.6%	0.12	3.58	9.53
Approve, later payout	48,234	53,268	+10.4%	0.07	2.18	5.81
Request paid instantly	–	56,383		0.08	2.30	6.15
Reject	29,222	26,449	−9.5%	0.04	1.08	2.88

C. Validating gas measurement against Sepolia

Sepolia receipts give gas used as fee divided by gas price. Table IV compares them with the same calls on the local v1.0 build. Registration and withdrawal requests match within the rounding of the explorer's fee display. Approvals are 0.5–0.6% higher locally; the deployed `RewardMechanism` is unverified, so the repository source may not be byte-identical to it, and function dispatch order alone can shift gas by a few hundred units. We take the local environment as accurate to within 1% for the comparisons that follow.

D. Gas: v1.0 versus v1.1

Table V compares the two versions. Registration costs 3.5% more in v1.1 because it verifies an ECDSA signature and stores a reverse mapping, which buys the identity binding. A single-user XP sync costs 33.7% less than v1.0's `updateUserXP` although it also computes the level, because the profile is one packed slot; with a level-up it costs 42,096 gas. In batches of 100 the cost falls to 14,004 gas per user (Fig. 2). A withdrawal request is 18.0% cheaper. The first payout to a user costs 34.6% more because it creates `claimed[user]`, a zero-to-nonzero store of about 22,100 gas; this is the price of moving v1.0's `localStorage` entry on chain, and it is paid once per user. Deploying v1.1 takes 3.81 M gas against 4.79 M for v1.0 (excluding v1.0's separate treasury funding transfer), because `RewardMechanism` creates and funds UCoin in its constructor.

The cost model differs from per-event designs in a useful way. A sync carries a *count*, so its gas does not depend on how many contributions the count covers. A user who made 30 contributions since the last sync costs the same 14,004 gas in a batch as one who made one. Deduplication is free because the monotonic rule replaces a per-contribution "seen" map, which in a per-event design costs a 22,100-gas store for every contribution.

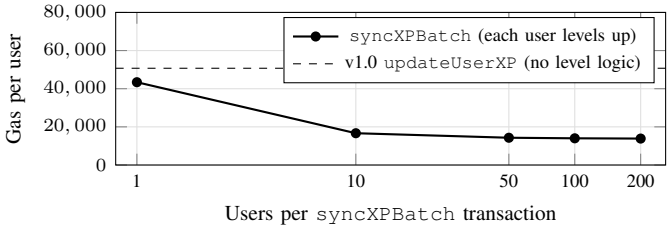


Fig. 2. Per-user gas of a batched XP sync. Every user in the batch crosses one level.

TABLE VI
500 USERS WITH NEW CONTRIBUTIONS AT ONCE

Mode	Txs	Total gas	Gas/user	Blocks
One `syncXP` per user	500	16,816,984	33,634	1
`syncXPBatch` of 100	5	5,218,883	10,438	1

E. Burst throughput

We registered 500 users, gave each some history, then let all 500 gain five contributions at once (no level is crossed) and drained the mempool with automatic mining off (Table VI). Submitting one `syncXP` per user used 16.8 M gas; five `syncXPBatch` calls of 100 users used 5.2 M, 69% less, or 2,088 gas per contribution. Both fit in one block at 30, 36 and 60 M gas limits, and every user ended at the same XP and level in both modes. If the system may use 5% of a 60 M-gas block (3 M gas per 12 s slot), batched syncs cover about 287 users per slot, so a six-hourly sweep of 100,000 active users needs roughly 350 slots, or 70 minutes.

F. Latency

Off-chain work is small. On the test machine, signing a binding attestation takes 6.0 ms on average (p95 8.1 ms), parsing a four-year GraphQL response 0.01 ms, and encoding and signing a 100-user batch transaction 8.7 ms (p95 11.4 ms). The GitHub API round trip, which we could not measure here, and slot timing dominate. Ethereum produces a block every 12 s [37]; a uniformly timed submission waits on average half a slot plus the propagation margin, about 7–11 s to inclusion in a Monte-Carlo slot model with 1–3 s submission delay and up to 5% empty slots, and economic finality takes about 12.8 minutes. A user who presses "Sync now" therefore sees new XP in roughly one GitHub round trip plus one slot. Users who do nothing are updated by the six-hourly sweep, so the worst-case staleness of passive users is six hours plus the sweep time; v1.0, by contrast, never updated on-chain XP at all.

G. Treasury capacity

Table VII and Fig. 3 apply Proposition 3. v1.0's supply would have paid level 20 to about 2,800 contributors and level 100 to fewer than five; the treasury would have run dry at the scale of a single mid-sized open-source community. With

TABLE VII
PROGRESSION AND NUMBER OF FUNDABLE USERS

					Users fundable to L	
L	**XP**	**Contrib.**	$S(L)$ **UCN**		**v1.0 (0.7 M)**	**v1.1 (70 M)**
1	100	2	5.4		129,630	12,962,963
5	1,500	30	31.7		22,096	2,209,621
10	5,500	110	78.2		8,948	894,827
20	21,000	420	247.1		2,833	283,269
50	127,500	2,550	3,098.4		226	22,593
100	505,000	10,100	148,416.4		4.7	471.6

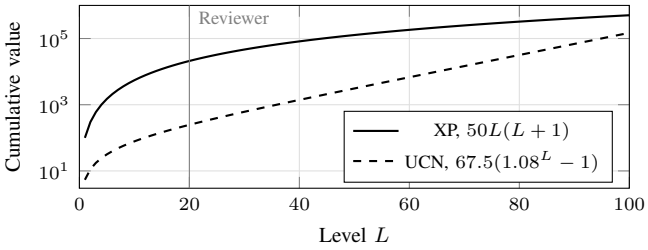


Fig. 3. Cumulative XP and cumulative UCN by level; the vertical line marks Reviewer promotion at level 20.

10^8 UCN, v1.1 carries 283,269 contributors to the Reviewer level.

V. DISCUSSION AND LIMITATIONS

A. What the prototype taught us

Three lessons from v1.0 generalise to other incentive DApps. First, a limit enforced only in the user interface is not a limit: F2 and F6 both come from treating the dashboard's arithmetic as the system's rules. Second, a role that no code path exercises (`XP_UPDATER_ROLE`) is a sign that a component is missing, and the Sepolia history confirms it. Third, wherever a number is shown to users, it should be the number the contract will act on; v1.0's leaderboard, dashboard and contracts each had their own idea of a user's standing.

B. Storage and decentralisation

Keeping XP and entitlements on chain buys auditability and lets anyone rebuild the leaderboard from events. It also makes each sync cost gas, which at 15–40 gwei on L1 is \$0.57–1.53 per user even in batches. Two moves would cut this by one to two orders of magnitude without changing the contracts: deploying them on an optimistic or validity rollup [38], [39], where most Ethereum activity already runs [36], and posting a Merkle root of all counts per sweep, with per-user proofs submitted only when a user withdraws. Rich metadata such as streaks and review text belongs in an indexer or on IPFS [40].

C. Quality versus quantity

A fixed 50 XP per contribution makes XP a count, and counts invite Goodhart's law [41]. Commit splitting, trivial issues, self-reviews in throwaway repositories and duplicate GitHub accounts all raise the count. Proposition 4 sharpens the concern: the yield past level 13 rises to 55 UCN per contribution at level 100, a level that takes an honest contributor decades and a script days. GitHub data is also a noisy measure of work [42], and visible counters change when people work [26]. v1.1 does not solve this. It makes counts public and reproducible, so abuse is visible, and it gives administrators suspension and payout review. Future versions should weight contributions by review outcome rather than count them [43], [44], cap XP per period, index the reward schedule to the remaining treasury, and move these parameters to DAO governance [45], [46]. A token's value also depends on adoption, which a schedule alone does not create [47].

D. Threats to validity

Our v1.0 evidence is a seven-user testnet deployment; it shows what the system did, not how a large community would behave. The 0.5–0.6% gap in approval gas may come from a deployed bytecode that differs from the repository. GitHub responses in backend tests are fixtures, and we did not measure GitHub API latency. Contribution counts follow GitHub's own counting rules, which, for example, credit commits only on default branches. We ran no user study and make no claim that XP, levels or titles change contribution behaviour [22], [23].

VI. CONCLUSION AND FUTURE WORK

Udyamita 1.0 demonstrated the idea of turning GitHub activity into on-chain levels and tokens, and its Sepolia record shows a team using it. It also showed where prototypes of this kind fail: the browser held the rules, the contracts held the money, and nothing connected them. Version 1.1 moves identity, XP, levels, entitlements and claims on chain, binds one GitHub account to one wallet, and replaces client-side XP with a monotonic, publicly reproducible count. The contract suite passes 71 tests with full coverage, local gas agrees with Sepolia within 0.6%, batched syncs cost 14,004 gas per user, and a 10^8-UCN supply funds a hundred times more contributors than v1.0's.

Next steps follow from the limitations: deploy v1.1 on Sepolia and then on an L2; replace the single attester with several independent oracle nodes or authenticated TLS data [48], so that no one key decides a count; weight XP by review outcome; add GitLab and Bitbucket through a forge prefix on the account id; and put reward parameters and payout review under DAO control.

REFERENCES

[1] M. Hoffmann, F. Nagle, and Y. Zhou, "The value of open source software," Harvard Business School, Working Paper 24-038, 2024.

[2] F. Nagle, D. A. Wheeler, H. Lifshitz-Assaf, H. Ham, and J. L. Hoffman, "Census II of free and open source software—application libraries," The Linux Foundation and Harvard Laboratory for Innovation Science, Tech. Rep., 2022.

[3] J. Lerner and J. Tirole, "Some simple economics of open source," *J. Industrial Economics*, vol. 50, no. 2, pp. 197–234, 2002.

[4] N. Eghbal, "Roads and bridges: The unseen labor behind our digital infrastructure," Ford Foundation, Tech. Rep., 2016.

[5] ——, *Working in Public: The Making and Maintenance of Open Source Software*. Stripe Press, 2020.

[6] Z. Durumeric *et al.*, "The matter of Heartbleed," in *Proc. ACM Internet Measurement Conf. (IMC)*, 2014, pp. 475–488.

[7] R. Hiesgen, M. Nawrocki, T. C. Schmidt, and M. Wählisch, "The race to the vulnerable: Measuring the Log4j shell incident," in *Proc. Network Traffic Measurement and Analysis Conf. (TMA)*, 2022.

[8] N. Raman, M. Cao, Y. Tsvetkov, C. Kästner, and B. Vasilescu, "Stress and burnout in open source: Toward finding, understanding, and mitigating unhealthy interactions," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.: New Ideas and Emerging Results (ICSE-NIER)*, 2020.

[9] C. Miller, D. G. Widder, C. Kästner, and B. Vasilescu, "Why do people give up FLOSSing? A study of contributor disengagement in open source," in *Open Source Systems (OSS 2019), IFIP AICT, vol. 556*. Springer, 2019.

[10] I. Steinmacher, M. A. G. Silva, M. A. Gerosa, and D. F. Redmiles, "A systematic literature review on the barriers faced by newcomers to open source software projects," *Information and Software Technology*, vol. 59, pp. 67–85, 2015.

[11] C. Overney, J. Meinicke, C. Kästner, and B. Vasilescu, "How to not get rich: An empirical study of donations in open source," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, 2020.

[12] N. Shimada, T. Xiao, H. Hata, C. Treude, and K. Matsumoto, "GitHub Sponsors: Exploring a new way to contribute to open source," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, 2022.

[13] Y. Zhauniarovich, Y. Boshmaf, H. A. Jawaheri, and M. A. Sabah, "Characterizing Bitcoin donations to open source software on GitHub," arXiv:1907.04002, 2019.

[14] J. Zhou, S. Wang, C.-P. Bezemer, Y. Zou, and A. E. Hassan, "Studying the association between Bountysource bounties and the issue-addressing likelihood of GitHub issue reports," *IEEE Trans. Softw. Eng.*, vol. 47, no. 12, 2021, arXiv:1904.02724.

[15] Dework, "Dework: Web3-native task management and bounties," https://dework.xyz, 2023.

[16] V. Buterin, Z. Hitzig, and E. G. Weyl, "A flexible design for funding public goods," *Management Science*, vol. 65, no. 11, pp. 5171–5187, 2019.

[17] Gitcoin, "Gitcoin grants stack and passport documentation," https://www.gitcoin.co, 2024.

[18] SourceCred, "SourceCred documentation: How cred works," https://sourcecred.io/docs, 2021.

[19] Coordinape, "Coordinape documentation: GIVE circles and epochs," https://docs.coordinape.com, 2023.

[20] J. O. Silva, I. Wiese, D. M. German, C. Treude, M. A. Gerosa, and I. Steinmacher, "A theory of the engagement in open source projects via summer of code programs," in *Proc. 28th ACM Joint Eur. Softw. Eng. Conf. and Symp. Found. Softw. Eng. (ESEC/FSE)*, 2020.

[21] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining "gamification"," in *Proc. 15th Int. Academic MindTrek Conf.*, 2011, pp. 9–15.

[22] J. Hamari, J. Koivisto, and H. Sarsa, "Does gamification work? A literature review of empirical studies on gamification," in *Proc. 47th Hawaii Int. Conf. System Sciences (HICSS)*, 2014, pp. 3025–3034.

[23] A. Anderson, D. Huttenlocher, J. Kleinberg, and J. Leskovec, "Steering user behavior with badges," in *Proc. 22nd Int. Conf. World Wide Web (WWW)*, 2013, pp. 95–106.

[24] L. Mamykina, B. Manoim, M. Mittal, G. Hripcsak, and B. Hartmann, "Design lessons from the fastest Q&A site in the west," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, 2011, pp. 2857–2866.

[25] O. Pedreira, F. García, N. Brisaboa, and M. Piattini, "Gamification in software engineering—A systematic mapping," *Information and Software Technology*, vol. 57, pp. 157–168, 2015.

[26] L. Moldon, M. Strohmaier, and J. Wachs, "How gamification affects software developers: Cautionary evidence from a natural experiment on GitHub," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, 2021.

[27] E. G. Weyl, P. Ohlhaver, and V. Buterin, "Decentralized society: Finding Web3's soul," SSRN 4105763, 2022.

[28] R. Bloemen, L. Logvinov, and J. Evans, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, https://eips.ethereum.org/EIPS/eip-712, 2017.

[29] M. Wöhrer and U. Zdun, "Smart contracts: Security patterns in the Ethereum ecosystem and Solidity," in *Proc. Int. Workshop on Blockchain Oriented Software Engineering (IWBOSE)*, 2018.

[30] G. Caldarelli, "Understanding the blockchain oracle problem: A call for action," *Information*, vol. 11, no. 11, p. 509, 2020.

[31] J. R. Douceur, "The Sybil attack," in *Peer-to-Peer Systems (IPTPS 2002), LNCS, vol. 2429*. Springer, 2002, pp. 251–260.

[32] D. Siddarth, S. Ivliev, S. Siri, and P. Berman, "Who watches the watchmen? A review of subjective approaches for Sybil-resistance in proof of personhood protocols," *Frontiers in Blockchain*, vol. 3, p. 590171, 2020.

[33] GitHub, "GitHub REST, GraphQL and webhooks documentation," https://docs.github.com, 2026.

[34] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," in *Principles of Security and Trust (POST 2017), LNCS, vol. 10204*. Springer, 2017, pp. 164–186.

[35] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2016, pp. 254–269.

[36] ethereum.org, "Building on Ethereum in 2026: What has changed," https://ethereum.org/latest/building-on-ethereum-in-2026, 2026.

[37] V. Buterin, D. Hernandez, T. Kamphefner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, and Y. X. Zhang, "Combining GHOST and Casper," arXiv:2003.03052, 2020.

[38] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in *Proc. 27th USENIX Security Symp.*, 2018, pp. 1353–1370.

[39] L. T. Thibault, T. Sarry, and A. S. Hafid, "Blockchain scaling using rollups: A comprehensive survey," *IEEE Access*, vol. 10, pp. 93 039–93 054, 2022.

[40] J. Benet, "IPFS—content addressed, versioned, P2P file system," arXiv:1407.3561, 2014.

[41] M. Strathern, "'Improving ratings': Audit in the British university system," *European Review*, vol. 5, no. 3, pp. 305–321, 1997.

[42] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proc. 11th Working Conf. Mining Software Repositories (MSR)*, 2014, pp. 92–101.

[43] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, 2013, pp. 712–721.

[44] G. Gousios, M. Pinzger, and A. van Deursen, "An exploratory study of the pull-based software development model," in *Proc. 36th Int. Conf. Softw. Eng. (ICSE)*, 2014, pp. 345–355.

[45] S. Hassan and P. De Filippi, "Decentralized autonomous organization," *Internet Policy Review*, vol. 10, no. 2, 2021.

[46] S. Wang, W. Ding, J. Li, Y. Yuan, L. Ouyang, and F.-Y. Wang, "Decentralized autonomous organizations: Concept, model, and applications," *IEEE Trans. Computational Social Systems*, vol. 6, no. 5, pp. 870–878, 2019.

[47] L. W. Cong, Y. Li, and N. Wang, "Tokenomics: Dynamic adoption and valuation," *Review of Financial Studies*, vol. 34, no. 3, pp. 1105–1155, 2021.

[48] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, "Town crier: An authenticated data feed for smart contracts," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2016, pp. 270–282.